

Formats & stockage de données

CSV / Parquet / base : choisir selon qui consomme la donnée · Marianne A.

L'idée. On ne choisit pas un format « par défaut » — on le choisit selon **QUI** ou **QUOI** consomme la donnée. Trois usages, trois réflexes : **CSV** pour échanger avec un *humain*, **Parquet** pour faire circuler la donnée *entre traitements*, **base SQL** pour *interroger* / *mettre à jour* des données vivantes.

Règle Un dataset intermédiaire relu par du **CODE** (et pas par un humain) → réflexe **.parquet** , pas **.csv** .

1 Les trois (quatre) usages : à quoi sert quel format

TU VEUX...	FORMAT	POURQUOI
Qu'un humain ouvre / édite le fichier (Excel, mail, debug).	CSV	Universel, lisible à l'œil — mais non typé (tout est du texte).
Faire circuler de la donnée entre traitements (étape N → N+1).	Parquet	Typé, compact, rapide ; en revanche binaire, <i>illisible à l'œil</i> .
Interroger / mettre à jour des données vivantes.	Base SQL (SQLite, PostgreSQL)	Requêtes, intégrité, transactions.
Analytique à très grande échelle .	Entrepôt / data lake (Spark, DWH)	Culture générale ; souvent <i>hors périmètre</i> à petite échelle.

2 CSV vs Parquet : ligne vs colonne

CSV — orienté LIGNE

ligne 1 : id, nom, montant

ligne 2 : 1, a, 100

ligne 3 : 2, b, 250

une ligne = un enregistrement complet

Parquet — orienté COLONNE

id : [1, 2, 3]

nom : [a, b, c]

montant : [100, 250, 90]

une colonne = un bloc de même type

Trois gains de **Parquet** découlent de ce stockage par colonne : **(1) types préservés** — le *schéma voyage avec la donnée*, on relit exactement ce qu'on a écrit ; **(2) compression 3–5×** — des valeurs de même type et souvent proches se compressent bien ; **(3) lecture sélective** — on ne lit que les colonnes utiles, sans parcourir tout le fichier.

Nuance technique. À la relecture d'un CSV, pandas *ré-infère* quand même int/float ; ce qui se perd vraiment, ce sont les types **sans équivalent texte** : dates, catégorielles, booléens, entiers *nullable*. Et sur de **très petits fichiers**, le gain de compression est modeste (overhead de métadonnées) — c'est surtout le **typage** qui justifie Parquet.

3 Quand choisir quoi : table de décision

SITUATION	CHOIX	RAISON
Livrable à relire / éditer par un humain.	CSV	Lisible partout, ouvrable dans un tableur.
Fichier intermédiaire relu par du code (features, cache).	Parquet	Types préservés, compact, relecture fidèle et rapide.
Besoin de requêtes, jointures, mises à jour ciblées.	Base SQL	Le moteur fait le travail : filtres, index, intégrité.
Plusieurs utilisateurs en écriture concurrente.	PostgreSQL (pas SQLite)	Concurrence multi-writers gérée par le serveur.

4 Pièges & idées reçues

IDÉE REÇUE / PIÈGE	LA RÉALITÉ
« Parquet > CSV toujours »	Non : trois usages distincts . On choisit selon le <i>consommateur</i> de la donnée, pas par principe.
« CSV garde mes types »	Non : dates, catégories et booléens retombent en texte ; à la relecture il faut les reconstruire.
« SQLite gère la concurrence multi-utilisateurs »	Non : un seul writer (verrou global). SQLite sert pour requêtes / intégrité / mises à jour ciblées, <i>pas</i> le multi-accès. Vrai multi-accès → PostgreSQL .
« Il me faut Spark / un DWH »	Souvent sur-ingénierie à petite échelle. Rester simple tant que le volume ne l'impose pas.

La règle d'or. CSV pour échanger avec un humain · **Parquet** pour faire circuler de la donnée entre traitements · **base** pour interroger / mettre à jour. Choisir selon le *consommateur* de la donnée, pas par défaut.

À retenir. Le bon format n'est pas le plus « moderne » mais celui qui colle au consommateur : humain → CSV, code → Parquet, données vivantes → base. Le typage (pas la seule compression) est ce qui fait pencher pour Parquet.